

Lab 6: Single Point Positioning (SPP)

In this lab we will try to implement a single point positioning (SPP) solution – the simplest GPS positioning solution possible. In this case we will use only the GPS L1 code pseudoranges to keep it simple. We will account for satellite clock errors and to some degree the satellite orbit errors, but nothing more.

Initially you need to:

1. Download the lab from Learn, including the lab description, the Python files, and the sp3 file which contains rapid orbits from the day of the field work
2. Put your own field data in the same folder as the lab files

Now, open the lab6.py file in your favorite IDE. This file contains the code template that you need to work with. Start by looking through the file.

Then, replace file paths to load your own data and the new sp3 file into Python. Start by using the data from the first session of the field work; the good data.

Your task is now:

1. Finish the implementation of the function *create_kernel()*. You will need to:
 - a. Determine ranges between receiver and satellites, *rng1*
 - b. Use those ranges to compute unit vectors between receiver and satellites, *a1*
2. Next you will need to implement the actual non-linear least squares (NLLS) solution in the function *spp()*. Remember the NLLS solution is:

$$\Delta x = (A^T A)^{-1} A^T L$$

TIPS and important notes:

Note that your solution, *x*, in the *spp* file must be a numpy array of size 4.

ALSO, be aware, that when you perform mathematical operations on pandas DataFrames you need to ensure that they have the same indexes. In some cases it will make sense to convert your DataFrames or Series to numpy arrays (for instance using *.values* command).

If you need to add a numpy array with only one dimension to a DataFrame with two dimensions, you can use `array[:, None]` to expand the array to two dimensions.

For an example, see line 148 in the template code, where satellite clock error is added to the pseudoranges.

Once you've completed the implementation you must do the following:

1. Use matplotlib in Python to create a 3D scatter plot of your solutions *and* the “true” position of the receiver found in the header of the RINEX file (Please note that this is not actually a true solution – but in lack of better we will use it as a reference for now)
2. Discuss/consider your solution: Does it seem reasonable? Why is it not better / worse?
3. Plot the clock error of the solution to see how the receiver clock drifts with time – does it look as expected?

4. Use the function `dataframe_to_kml()` to convert your solution into a kml file. Load this solution into Google Earth along with the kml from the receiver data. How does your solution compare to the receiver solution?

Now, let's look at the DOP and uncertainty of the solutions:

1. First, determine PDOP for each epoch
2. Next, determine the sum of squared residuals for each epoch
3. Combine these to find the total variance of the positioning solution, σ_{pos}^2
4. Finally, discuss the uncertainties – what is the quality of the solution?

Once that is done, we should also look at both the bad session and the kinematic/moving session:

1. Compute SPP solutions for both other sessions.
2. Convert them to kml and again try to compare them with what you got from the receiver.

Finally, if you have the time, we should try to improve on our positioning solution. We will not account for atmospheric errors or multipath, as you will work with this later in the course. But there are a few things we might try.

You are welcome to try to implement the following suggestions in any which order you like.

1. Try to use your previously calculated position as input to the next epoch (basically set $x_0=x$) to provide a better initial guess for the NLLS solution – this will not improve the SPP solution itself but should speed up the computations.
How much faster does the code now run?
2. In lecture 2 we discussed using the receiver clock error to compute more accurate estimates of the signal travel time, τ . Use the determined receiver clock error to recompute τ , satellite positions AND receiver position for each epoch.
How much does receiver position change (improve)?
3. Use your positioning solution to compute ENU coordinates of all satellites for each epoch. Now, implement an elevation mask (10 or 15 degrees), and compute the SPP using only satellites above that mask.
How much does the receiver position change?